

Step 1: Understanding the Dataset

Before you clean or prepare data for a machine learning model, the first and most important step is to understand the dataset. This step tells you *what the data contains, how it is structured, and what problems you might need to fix later*

- Load the Dataset
- Take a First Look (Preview Rows)
- Identify Feature Types
 - What kind of information does each row represent?
 - Do the columns make sense?
 - Are there obvious errors?
- Understand the Target Variable (Label)
- Look for Basic Statistics
- Look for Data Quality Issues

Step 2: Identify Important Columns

- Categorize each column as Numeric, Categorical, Text, Date/Time, ID or Identifier. IDs are almost always dropped because they don't help the model learn patterns. Examples of ID-like columns: `customer_id`, `user_id`, `transaction_id`, `serial_number`
- Every column should be checked for uniqueness count. This tells whether a feature is useful or useless. Rule of thumb: A column with 100% unique values (all values different) do NOT help prediction so drop it. Similarly A column with only 1 value (same value for all rows) has zero information so drop this too
- Sometimes two or more columns carry the same information. Eg. `birthdate` and `age`, `first_purchase_date` and `join_date`, `country` and `location_country`. Keep the most useful one and drop duplicates or less accurate versions.
- If a column has too many missing values, it may not be worth saving. Rules of Thumb:
 - **> 60% missing**: Likely drop
 - **30–60% missing**: Depends on importance
 - **< 30% missing**: Usually keep and impute
- Drop columns that do not help the model. Eg. Free-form text like “notes”, HTML content, Image URLs, Metadata like “`last_updated_by_user`”, Operational columns like “`file_import_timestamp`”
- Drop leakage columns: Data leakage means the column contains information that would not be available at prediction time. Example:
 - `final_price_after_discount` if trying to predict `final_price`
 - `disease_confirmed` if predicting whether someone is sick
 - `outcome` if you are predicting the outcome

- Remove one of the correlated columns. Create a correlation matrix for numeric columns to identify highly correlated pairs (0.85+) eg, height_cm and height_m or income_monthly and income_yearly

Step 3: Handling Missing Values

Missing values (NaN, null, blank cells) must be handled before any model training. Models cannot directly work with missing data.

- Drop a column if Missing percentage > **60%**
- Drop rows if:
 - The dataset is large
 - Only a few rows contain missing values
 - Dropping won't lose important information
- Impute (Fill) Missing Values if Column Is Important:
- Numeric Columns
 - Mean Imputation (Good for normal data)
 - Median Imputation (Better for skewed data)
 - Zero/Constant Imputation if zero means something (e.g., no purchases) or you explicitly want to mark "missing"
 - Model-Based Imputation (advanced)...eg use KNN or Regression for imputation
- Categorical Columns
 - Mode Imputation
 - "Unknown" or "Other" Category to represent missing intentionally.
 - Forward-fill: Use last known value/ Backward-fill: Use next known value for time Series Data
- Date/Time Columns
 - Replace missing dates with: Most frequent date/ Mean date/Earliest date/ A special placeholder like "1900-01-01"
 - Or add another columns for example: date_missing_flag = 1 if date is missing, else 0

After Imputation, Recheck the Dataset:

- No missing values remain (unless intentionally left)
- All filled values are reasonable
- No new errors were created

Step 4: Handling Duplicate Data

Duplicate rows (or near-duplicates) can negatively impact the model because they:

- Overrepresent certain cases
- Bias averages and statistics
- Cause the model to “see” the same pattern multiple times
- Reduce model generalization

To identify duplicate data: `df.duplicated().sum()`

If duplicates exist, delete them.

Step 5: Handling Outliers

Outliers can distort model training, affect averages, break scaling, and cause poor performance. An outlier is a value that is:

- Extremely high or low compared to the rest
- Impossible or unrealistic
- A data entry mistake
- A rare event that might not reflect normal behavior

Only numeric columns typically have outliers. To detect outliers use:

- IQR method (only use it when dataset is not normally distributed because we don't want to unnecessarily sort a large dataset)
- Z-Score Method (For normally distributed data)
- Or visualizations: Box Plot/Histograms/Scatter Plots

To handle outliers:

- Remove Outliers only when:
 - Outlier is a mistake
 - Outlier is impossible
 - Outlier severely affects model
 - Very few outliers exist
- Cap Outliers (Winsorization): replace extreme values with Lower outlier (value at 1st percentile)/ Upper outlier(value at 99th percentile). Use it when you dont want to lose data or values are rare
- Use transformations to reduce the influence of extreme values. eg, $\log(x)$ / $\sqrt{x}$ / power transforms

Step 6: Fixing Inconsistent and Incorrect Data

Models cannot learn correctly from messy or incorrect data!

- Fix Inconsistent Formatting in Categorical Columns eg,
 - Different capitalization
 - Extra spaces
 - Misspellings
- Standardize Units: check whether values use different units, which breaks consistency eg Height in cm vs. meters
- Fix Incorrect or Impossible Values eg, Age = 500/ Weight = -20/Salary = -300/Height = 0 cm
- Dates often come in different styles which causes confusion and errors. Convert all dates into one format
- Sometimes columns have the wrong type. eg,
 - "35" stored as a string instead of number
 - "2020-01-03" stored as text instead of date
 - "100,000" stored with a comma
- Some categories appear only a few times and are not helpful for modeling. Replace rare categories with "Other".

We did not practice this step much in class so only apply it in exam where important. This was just for information.

Step 7: Encoding Categorical Variables

Categorical variables are columns that contain labels, names, or categories instead of numbers. ML models require numerical input, so all categorical values must be encoded.

- One-Hot Encoding used for Nominal categories (no order)
- Label Encoding for binary categorical labels
- Ordinal Encoding you want to preserve an order

DO NOT use Label encoding on nominal categories because the model will think: red (1) < blue (2) < green (3) which makes no sense.

- Frequency Encoding: Replace each category with how often it appears. useful when we have too many categories to one-hot and categorical column is important (not covered in class just for information, you can stick to first 3 for exam)

Encoding must be done after train-test split, not before because if you calculate target encoding using the entire dataset, the model “sees” the answer from the test set, this is cheating. So correct order is (1) Split into train & test (2) Fit encoders on training data (3) Apply encoders to test data

Step 8: Scaling / Normalization

This step ensures that numeric features are brought to the same scale, so machine learning models can learn properly. Many algorithms perform poorly if features have very different ranges.

Not all columns need scaling. Only numeric continuous features. Do NOT scale categorical encoded features/ binary columns (0/1)/ One-hot-encoded columns

We only covered 2 types of feature scaling in class

- Standardization (Z-Score Normalization): Not good for data that is heavily skewed or has many outliers.
- Min–Max Scaling (Normalization): When all features must be on the same small scale. Not good for data with outliers (min-max gets distorted)

Once again scaling must be applied after the train-test split, not before. If you scale before splitting, the test data will “learn” information from the training distribution and this is data leakage.